DAY 24

Databricks Autoscaling & Cost Optimization

Performance Bhi, Paisa Bhi Save

Core idea: workload badhe toh compute badhao, workload kam ho toh compute reduce karo, aur idle compute ko unnecessarily running mat rakho.

1. The Problem with Fixed-Size Clusters

- In a fixed-size cluster, the number of workers stays constant even when workload demand changes.
- Example: a cluster has 10 workers, but a light workload needs only 2. The remaining workers may be idle while compute charges continue.
- This is over-provisioning: capacity is sized for peak demand even though peak demand may exist only briefly.

2. What Autoscaling Does

Instead of selecting one fixed worker count, define a minimum and maximum range:

Setting	Example
Minimum workers	2
Maximum workers	10

Databricks can then add or remove workers within that range as demand changes. Example: 2 → 5 → 10 → 4 → 2 workers.

3. Autoscaling vs Auto Termination

Concept	What it changes	When it helps
Autoscaling	Worker count inside a running cluster	When workload demand changes
Auto Termination	The entire idle compute resource	When the cluster is no longer being used

Mental model: Autoscaling = “kitne workers chahiye?” Auto Termination = “compute ko ab chalna bhi chahiye ya band hona chahiye?”

4. Job Compute for Scheduled ETL

- Interactive/all-purpose compute is useful for development and exploration, but keeping it running for scheduled production ETL can waste money.
- Job Compute can be created when a scheduled job starts and terminated when the job finishes.
- This aligns compute runtime more closely with actual workload execution.

5. Serverless Compute

For supported workloads, serverless compute reduces the infrastructure sizing and cluster-management work you handle manually. Databricks manages much of the underlying compute lifecycle and scaling behavior.

Simple mental model: Classic compute exposes more explicit cluster controls; serverless shifts more infrastructure management to Databricks.

6. Cost Optimization Levers

- **Right-size compute:** Avoid provisioning much more CPU/memory than the workload needs.
- **Autoscaling:** Change compute capacity with workload demand.
- **Auto termination:** Stop idle compute instead of paying for unused runtime.
- **Job compute:** Use workload-scoped compute for scheduled ETL and batch jobs.
- **Spot / preemptible capacity:** Consider for interruption-tolerant workloads where appropriate.
- **Cluster policies:** Use guardrails to restrict expensive instance types, worker counts, and risky configurations.
- **Efficient runtime / engine:** Faster execution can lower total compute consumed.
- **Measure usage:** Identify the workloads actually driving spend before optimizing.

7. Track Spend with Billing System Tables

A useful starting point for usage analysis is Databricks billing system tables. Example:

```
SELECT *
FROM system.billing.usage;
```

Questions this can help answer: Which workloads consume the most DBUs? Which jobs run longest? Are clusters sitting idle? Did a configuration change increase spend?

8. Key Takeaways

Workload ↑ → Compute ↑ More demand, more capacity	**Workload ↓ → Compute ↓** Less demand, less capacity	**Idle → OFF** Do not pay for idle compute

Cost Optimization = Right Compute × Right Size × Right Runtime × Right Duration

9. Reel-Friendly One-Line Definitions

- **Fixed Cluster:** "Maine capacity pehle hi decide kar di."
- **Autoscaling:** "Workload ke according workers adjust karo."
- **Auto Termination:** "Kaam hi nahi hai toh compute band karo."
- **Serverless:** "Infrastructure aur scaling ka major headache Databricks handle kare."

10. Quick Revision

Why can a fixed cluster waste money? Because worker count remains fixed even when demand falls.

What does autoscaling change? The number of workers in a running cluster.

What does auto termination change? Whether the idle compute keeps running at all.

When is Job Compute useful? For scheduled, workload-scoped jobs such as production ETL.

Why track billing data? To optimize based on measured consumption instead of guesses.

Next Topic: Day 25 — Photon Engine

VISUAL SUMMARY

